TECHNICAL WHITE PAPER · VERSION 2

WYRM.LAB
A Deterministic Puzzle Laboratory for Auditing the Spatial Reasoning of Large Language Models



An original grid-worm game whose five mechanic families are engineered to defeat pattern-matching; a five-layer isolation model that never trusts an agent's claims; a control plane for pausing, interrupting, and steering models mid-run; and an experience layer that renders the machine's textual world beside a human engraving of the same deterministic state. Twelve BFS-certified levels, 1–10 difficulty, full Docker deployment.

The WYRM.LAB Instrument

Engine v2 · Agent Sandbox L1–L5 · Orchestrated Agentic Loop
Every level certified solvable · every thought recorded · every claim re-derived

KEYWORDS
language-model agents · spatial reasoning · agent isolation · deterministic game environments · Elo benchmarking · human–AI interaction

Contents

1 Abstract **2**

2 Introduction **2**

3 The WYRM Game **3**
3.1 State and dynamics 3
3.2 Mechanic families 3
3.3 The observation contract 4

4 Difficulty Engineering **4**
4.1 Certification 4
4.2 The difficulty score 5
4.3 The ladder 5

5 System Architecture **5**

6 Strict Agent Isolation **6**

7 The Operator Control Plane **7**

8 The Experience Layer **8**

9 Benchmark Methodology **8**

10 Implementation and Deployment **8**

11 Results and Observations **9**

12 Related Work **9**

13 Limitations and Future Work **9**

14 Conclusion **10**

1 Abstract

WYRM.LAB is a self-contained laboratory in which large language models (LLMs) play, generate, and critique levels of *WYRM*—an original, deterministic grid puzzle built around a segmented worm—while every byte of their experience is recorded, audited, and rendered for human inspection. The system contributes (i) a formal game specification spanning five mechanic families whose interactions—self-amputation, irreversible terrain, frictionless multi-cell slides, key inventory—are deliberately hostile to pattern-matching and demand explicit spatial simulation; (ii) a five-layer agent isolation model in which models receive only frozen text projections, produce strictly schema-validated replies, and never have their claims trusted, since a checker re-derives every outcome from the action log alone; (iii) an operator control plane supporting mid-run pause, interrupt, and steering over live token streams; and (iv) an experience layer that presents the machine's textual view and a human engraving-style rendering of the identical deterministic state side by side. A twelve-level difficulty ladder, each level certified solvable by exhaustive search with optimal solutions of 7–33 moves and validator-assigned difficulties of 1–10, grounds the benchmark; pairwise Elo updates rank agents after each encounter. The entire laboratory ships as a single Docker image with a persistent volume. We report the system architecture, the difficulty-engineering methodology, isolation design, and initial observations, including a partial LLM run whose recorded reasoning illustrates both the promise and the failure modes the instrument is designed to expose.

Keywords. language-model agents; spatial reasoning; deterministic game environments; agent isolation; benchmarking; Elo rating; human–AI interaction.

2 Introduction

Evaluating the *reasoning* of large language models has become inseparable from evaluating what they can *do*: agents now operate browsers, write code, and take actions in environments whose state they must understand, predict, and reshape. Yet the most widely used probes of reasoning are static benchmarks—mathematical word problems [1], multiple-choice science questions—in which a single answer is produced once and the trajectory that produced it is discarded. These instruments cannot expose *how* a model plans, only whether its final string matches. Game arenas addressed this in classical AI—chess, Go, StarCraft II [2–4]—by providing environments where every decision compounds, but the agents studied there were trained reinforcement-learners, not language models, and the environments were closed to human inspection of the agent's reasoning.

WYRM.LAB occupies the intersection. It is a Kaggle-Arena-style instrument in which LLM agents play an original puzzle game through a text interface, and *everything* is preserved: the exact observation shown before each decision, the raw completion, the parsed action, the environment's verdict, and the deterministic state that resulted. The design goal is not to crown a winner but to make reasoning *auditable*—to let an operator watch a model think its way through a maze, pause it mid-thought, steer it with a note, and later replay the run with mathematical certainty that what is being replayed is what happened.

Three commitments shape the system:

1. **Determinism as an epistemic foundation.** The game engine is pure: the same level and action sequence always yield the same trajectory, on the server that agents play against and in the browser that humans watch. This converts every recorded run into a reproducible artifact and makes the checker's certification a proof rather than an assertion.

2. **Text as the agent's world.** Models receive the board as ASCII, the rules as prose, and the status line as key–value pairs. Nothing pixel-level, nothing hidden: the interface is the specification, and the specification is small enough for a model to actually hold in context.
3. **Adversarial-by-design mechanics.** The mechanic families are chosen so that local greed fails: self-amputation shortens the body but leaves permanent obstacles; one-way valves and crumbling bridges make travel irreversible; ice turns one action into a multi-cell glide whose landing point, not its entry point, is what matters. These are precisely the structures on which pattern-matching collapses and explicit simulation survives [9].

The remainder of this paper describes the game formalization and its difficulty engineering (Sections 3–4), the system architecture (Section 5), the isolation model (Section 6), the operator control plane and experience layer (Sections 7–8), the benchmark methodology (Section 9), implementation and deployment (Section 10), initial results (Section 11), related work (Section 12), and limitations (Section 13).

3 The WYRM Game

3.1 State and dynamics

A level is a rectangular grid of characters; parsing it yields the initial game state

$$s \;=\; \big(T,\, C,\, A,\, K,\, W,\, R,\, P,\, h,\, t,\, L\big), \tag{1}$$

where T is the terrain matrix, C the multiset of crates, A the remaining apples, K the remaining keys on the ground, $W = (w_0, \ldots, w_{n-1})$ the ordered worm segments with head w_0, R the inert remains left by prior splits, P the set of collapsed bridge cells (a rendering hint; terrain already records them as rubble), $h \in \mathbb{N}$ the keys held, t the turn counter, and L the level's move budget. The transition function $\delta : S \times \mathcal{A} \to S \times \mathcal{E}$ maps a state and an action $a \in \mathcal{A} = \{\texttt{up}, \texttt{down}, \texttt{left}, \texttt{right}, \texttt{split}\}$ to a successor state together with an event list $\mathcal{E}$; it is pure, total (invalid actions return the same state with a rejection reason and consume no turn), and identical on server and client. A run is a sequence of applied actions; the win condition is the head reaching the exit with $A = \varnothing$.

3.2 Mechanic families

The terrain alphabet partitions into five families, summarized in Table 1. The *classic* families are conventional sokoban-like machinery: apples grow the worm (mandatory, since the exit stays sealed while any remain), crates are pushed and weigh down plates, plates open doors while every plate carries weight, dirt is dug through and permanently removed, spikes kill, and `split` cuts the worm in half—the front keeps the head, the back becomes inert, immovable, plate-weighting remains.

Table 1: Mechanic families and the reasoning load each imposes on an agent.

Family	Glyphs	Reasoning load
Classic	`* $ % D + ~`	sequential planning; growth–mobility trade-off; weighted logic
Split	`@ o :`	resource accounting of one's own body; permanent obstacle creation
Valves	`> < ^ v`	route irreversibility; loop construction to return
Bridges	`=`	consumable geometry; crossing-order commitment
Ice	`_`	multi-cell simulation from a single action; landing-point analysis
Keys	`k L`	inventory bookkeeping; detour sequencing

The four advanced families deserve comment, because they are the difficulty engine of the benchmark:

One-way valves. A valve tile may be entered only when the direction of entry matches the arrow. Standing on it and leaving is free. The consequence is that any crossing commits the agent to finding a *loop* back—an Eulerian-like obligation that punishes greedy apple collection order.

Crumble bridges. A bridge holds while occupied by any weight (worm, crate, or remains) and collapses into impassable rubble the instant the last weight leaves. Crossing is therefore a one-time act in *time* as well as space: an agent that crosses to fetch an apple it cannot otherwise reach has consumed geometry it may need for the return. Remains deliberately keep a bridge standing, creating a subtle interplay with `split`.

Ice. Entering a frictionless tile starts a slide: the head keeps moving in the entry direction until it lands on a non-ice cell or is stopped by an obstacle; the entire glide is one turn. Sliding into spikes is fatal; slides cannot dig or push. Formally, for entry cell c_0 and direction d,

$$\text{slide}(c_0, d) \;=\; c_m \quad \text{where} \quad m = \max\{\, k \le m_{\max} \;:\; T[c_i] = _ \;\forall\, i < k,\; c_{i+1} = c_i + d \text{ enterable} \,\}, \tag{2}$$

so the *landing* cell c_m, not the entered cell, is what an agent must predict. This is the single most LLM-hostile mechanic in the alphabet: a one-token action commits a multi-cell trajectory whose end state must be simulated exactly.

Keys and locks. Keys are entities picked up by walking; locks are terrain that consumes one key on entry and opens permanently. The obligation is inventory arithmetic—count the locks between here and the exit, gather that many keys *before* committing to a route that strands you without one.

3.3 The observation contract

The model-facing observation is a string of at most a few kilobytes:

```
CURRENT BOARD: {ascii grid}
STATUS: turn: t (limit L) | length: n | apples left: a | keys held: h |
plates weighted: x/y | door: OPEN/CLOSED | status: PLAYING
```

followed by an optional rejection note, the last eight actions with their validity, and an optional coach note. The legend and rules prose (roughly 1,400 tokens) travel in the system prompt. Every cell is one character; nothing is hidden, and nothing is pixelated—the interface *is* the game, which is precisely what makes failure interpretable: when a model dies on ice, the record shows whether it failed to parse the glyph, misremembered the rule, or simulated the glide incorrectly.

4 Difficulty Engineering

4.1 Certification

Every level—handcrafted, LLM-designed, or procedural—must pass the Logic Validator before entering the arena: an exhaustive breadth-first search over the full game graph proves solvability and yields a provably optimal (minimum-turn) solution. Search is two-phase: a first pass over the four movement actions only, falling back to a second pass including `split` when movement alone cannot win. The split action multiplies the reachable state space enormously (dropped remains can rest anywhere the body lay), so this ordering keeps certification tractable for the common case while remaining complete. State deduplication uses a canonical key

$$\kappa(s) \;=\; \big(W,\ \text{sort}(C),\ \text{sort}(A),\ \text{sort}(K),\ h,\ \text{sort}(R),\ [T]\big), \tag{3}$$

where $[T]$ denotes the terrain matrix itself, included only when mutable terrain (dirt, bridges, locks) exists—two states that differ only in dug dirt or collapsed bridges are distinct states. The search cap is 4×10^5 expanded states per phase.

4.2 The difficulty score

Difficulty is scored 1–10 by a transparent additive model over certified quantities:

$$D \;=\; \min\Big(10,\; \underbrace{\min(3, \lfloor m/10 \rfloor)}_{\text{solution length}} \;+\; \underbrace{\min(2, \lfloor \log_{10} N - 1 \rfloor)}_{\text{search breadth}} \;+\; \sum_{f} \beta_f \Big), \tag{4}$$

where m is the optimal length, N the states expanded, and β_f the family bonuses (split +2; crates +1; multiple plates +1; dirt ≥ 3: +1; ice ≥ 3: +1 and ice ≥ 6: +2; valves ≥ 3: +1; keys +1; bridges ≥ 3: +1). The score is deliberately mechanistic—it can be recomputed by anyone from the validator report—and deliberately conservative: it measures structural complexity, not estimated difficulty for any particular agent.

4.3 The ladder

Table 2 lists the twelve handcrafted levels of the shipped ladder with their certified properties. The ladder spans difficulty 1–10; the terminal *WYRM Trial* combines valves, bridges, ice, and keys under a 100-turn budget, and its 49,026-state search space reflects the compounding of irreversible structure. Turn budgets are tightened on advanced levels (70–110 versus the 120 default) so that thrashing—a characteristic LLM failure mode—exhausts the run even when death does not.

Table 2: The shipped difficulty ladder. D: validator difficulty; m^*: optimal solution length; N: BFS states expanded; TL: turn limit; families as in Table 1.

Level	Name	D	m^*	N	TL	Families
01	First Crawl	1	7	76	120	classic
02	Crate & Gate	4	11	1,290	120	classic
03	Thorn Run	2	10	258	120	classic
04	Self Service	5	14	5,176	120	classic (split required)
05	Growth Spurt	6	23	4,607	120	classic
06	The Gauntlet	5	22	268	120	classic
07	Key Note	6	33	2,000	70	keys
08	Slippery Slope	4	13	646	70	ice
09	No Return	5	29	5,490	90	valves
10	Burnt Bridges	5	24	393,139	80	bridges
11	Cold Ledger	8	16	196,731	90	ice + split + crates
12	WYRM Trial	10	33	49,026	100	valves + bridges + ice + keys

Two design observations from the table. First, difficulty is not length: *Cold Ledger* resolves in 16 moves yet scores 8, because its solution requires a split whose dropped half must come to rest on a pressure plate behind an ice field—a nine-move setup whose payoff is invisible to any policy that does not simulate. Second, the two largest search spaces (*Burnt Bridges*, *Cold Ledger*) arise where mutable terrain multiplies distinct states, which is exactly where LLM state-tracking is most fragile.

5 System Architecture

The laboratory is a single Next.js 16 application with a strictly layered interior (Figure 1). At the center sits the *deterministic core*: the pure game engine, the Logic Validator, the Checker,

and the Orchestrator that drives the sustained agentic loop around them. Wrapped around the core's agent-facing surface is the *sandbox*, the isolation boundary of Section 6. To the left, the *experience layer* renders the same recorded truth twice—once as the ASCII the model parsed, once as an ink-and-parchment engraving for humans—and hosts the Arena, the Forge level editor, and the live Control Room.

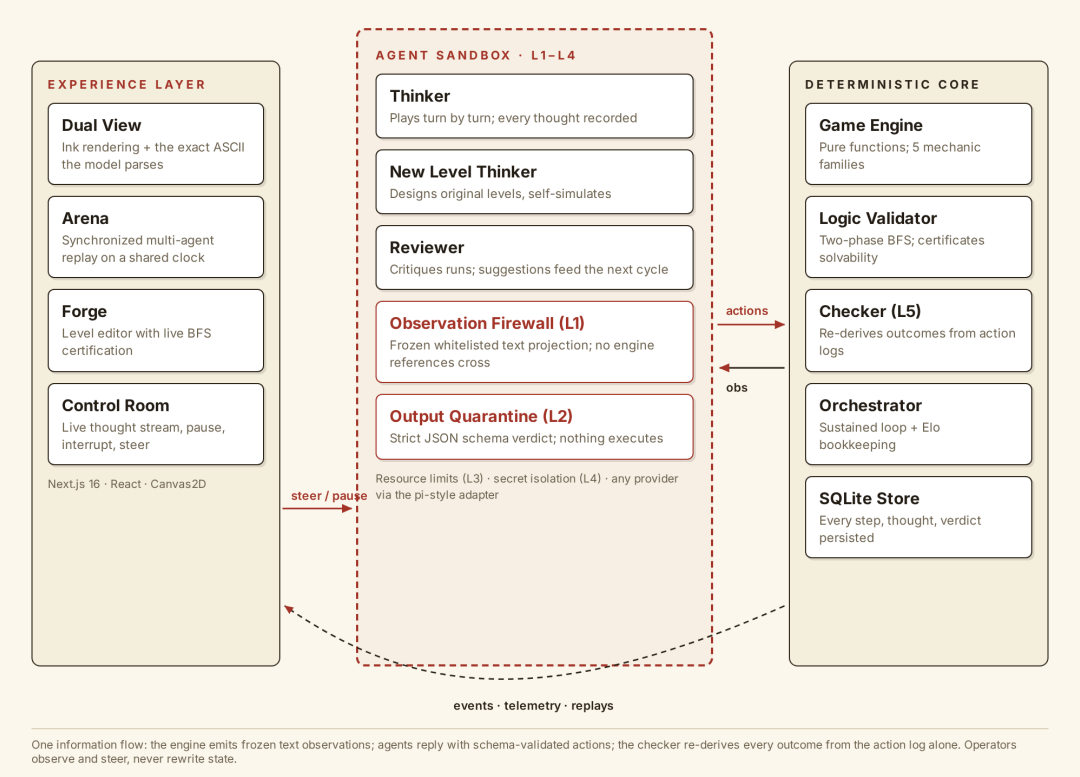


Figure 1: System architecture. Agents live behind the sandbox boundary (dashed); the deterministic core never trusts them, and the checker re-derives outcomes from action logs alone.

One *cycle* of the loop is: GENERATE (the New Level Thinker designs a candidate level, or a seeded procedural generator does) → VALIDATE (BFS certification; rejects are returned to the generator with the failure reason) → SOLVE (first the exhaustive baseline solver, establishing the par, then the LLM Thinker, with reviewer suggestions from the previous cycle injected as coach notes—the loop's memory channel) → CHECK (deterministic replay certification) → REVIEW (the Reviewer critiques the trajectory) → Elo update. Cycles chain autonomously; every artifact of every stage is persisted.

The provider layer follows the pi adapter philosophy: any endpoint—OpenAI-compatible, Anthropic, Google, OpenRouter, Groq, DeepSeek, Mistral, xAI, Together, Ollama, LM Studio, or a custom base URL—can be bound to each agent role at runtime from the Admin console, with per-role temperature and token budgets. A built-in model allows zero-configuration operation.

6 Strict Agent Isolation

Benchmarks that trust agent output measure compliance, not capability. WYRM.LAB's sandbox is specified as five layers, all enforced in code and all auditable after the fact (Table 3).

Layer 1 is implemented as a JSON round-trip of the whitelisted state fields followed by a deep freeze—a deliberately blunt construction whose virtue is that *only* declared primitive fields can survive it; no reference into engine internals can leak across a serialization boundary. Layer 2 quarantines the raw completion verbatim for the audit trail, then applies policy: 8,000-character

Table 3: The five isolation layers.

Layer	Enforcement	Guarantee
L1 Observation firewall	`projectObservation`	agents see a frozen, whitelisted text projection; engine references, database rows, filesystem and environment never cross
L2 Output quarantine	`parseActionReply`	raw replies are size-capped, control-stripped, schema-validated; malformed output is rejected with machine-readable violations; nothing executes
L3 Resource limits	call timeout, retry bounds	wall-clock and budget caps; operator interrupts propagate mid-stream
L4 Secret isolation	server-only provider rows	credentials never enter observations, logs, or client payloads (masked in the admin UI)
L5 Deterministic replay	Checker	outcomes are re-derived from (level, action log); claims are never trusted

cap, control-character stripping, and a strict schema requiring `action` to be one of the five verbs and `thought` a non-empty bounded string. Each model call appends a sandbox audit record—bytes in, bytes out, latency, verdict, violations—to the persisted step, so the isolation behavior of a run is itself data the dashboard can plot.

Layer 5 deserves emphasis because it inverts the usual relationship between agent and environment. The environment does not merely distrust the agent's *claims*; it distrusts the agent's *influence*. The only channel by which an agent affects the game is the five-verb action stream, and the checker replays that stream against a fresh engine instance. A run is certified exactly when the replayed terminal state matches the claimed one; any divergence marks the record invalid. Prompt-level manipulation of the harness is structurally impossible because there is no surface to manipulate: no tools, no function calling, no side-effectful parsing.

> **Design stance.** The agent is a *claim generator*; the environment is the *sole arbiter of truth*. Every interesting question about the agent—whether it understood the rules, simulated correctly, planned ahead—is answered by comparing its recorded reasoning against the deterministic consequence of its actions, never by asking it.

7 The Operator Control Plane

Each LLM run executes as a stepwise state machine whose loop can be held at any point between model calls. The control surface supports four operations: **watch** (a live event stream carries token deltas, applied actions, verdicts, and state snapshots over server-sent events), **pause** (the loop blocks at the next gate; the pause is recorded in the run's ledger), **interrupt** (an abort signal kills the in-flight provider call mid-generation and marks the run interrupted), and **steer** (free-text guidance queued into the next observation, marked as operator steering and counted in the run record). Steering is advisory—the note says *follow it unless it breaks a rule*—so a steered win remains a win, but the dashboard shows how much help the model received, and the Elo ledger counts interventions.

This machinery exists because mid-run intervention is where arena evaluation becomes laboratory experimentation: an operator who watches a model misread a valve can steer once and observe whether the correction propagates—a one-subject teachability probe embedded in an ordinary benchmark run.

8 The Experience Layer

The dashboard presents each run in two synchronized views. The *machine view* is the exact observation text, token-colored on a dark teleprinter panel—what the model parsed, not a stylization of it. The *human view* is a hand-drawn Canvas2D renderer that turns the same deterministic state into an ink-and-parchment engraving: cross-hatched bedrock, sepia stipple for dirt, timber bridges, a wax-red seal for the exit, and the worm as one continuous tapered ink creature with segment sutures, blinking eyes, and antennae that taste the air. Every event spawns ink—splashes when apples are eaten, debris when a bridge collapses, cold streaks on ice, a grey wash on death. The aesthetic is not decoration; it is the product thesis made visible: the laboratory is a *scientific instrument*, and instruments earn trust through craft.

Around the play view sit: the **Plays** archive (every run, its checker verdict, and its reviewer verdict); the **Arena** (two or three runs of the same level replayed side by side on a shared clock—one playhead, many minds); the **Levels** gallery (each level with live rendered preview, certified difficulty, and optimal play); the **Forge**, a level editor whose palette covers the full element vocabulary and whose validator runs BFS live as you paint; **The Loop** (cycle-by-cycle generator reasoning and validator reports); **Insights** (aggregate telemetry); the **Leaderboard**; and **Admin** (providers, isolation policy, asset governance). The design system is one consistent laboratory-paper language—warm paper ground, iron-gall ink, a single wax-red accent—chosen so that data reads as print, not glow.

9 Benchmark Methodology

Scoring. A run's score combines outcome and efficiency: $S = 100$ for a win, minus penalties for moves beyond the certified optimum and for deaths across attempts; the baseline solver, playing provably optimal lines, defines the par.

Rating. After each encounter on a level, every pair of models that ran that level is rated with the standard Elo update

$$R'_A \;=\; R_A + K\,(s_A - E_A), \qquad E_A \;=\; \frac{1}{1 + 10^{(R_B - R_A)/400}}, \tag{5}$$

with $K = 24$ and the expected-score term computed from the run scores rather than a binary win indicator, so a slow win outranks a fast loss only insofar as the scores say so. Ratings accumulate across levels and cycles; the exhaustive solver anchors the scale at provably optimal play.

Certification. The checker replays each run from (level, action log) on a fresh engine instance and compares terminal states; runs are labeled CERTIFIED or INVALID. Because the engine is pure and the level text is stored with the run, certification is repeatable by any third party—the audit trail is the benchmark.

10 Implementation and Deployment

The system is a single Next.js 16 application (React 19, TypeScript, Tailwind, Prisma over SQLite, TanStack Query, Framer Motion), with the engine shared isomorphically between server and client. The engine, validator, agents, and sandbox total roughly 2,400 lines of dependency-free TypeScript; the checker and baseline solver are direct consumers of the same `step` function the dashboard replays with, so no reimplementation drift is possible.

Deployment is a single Docker image: a three-stage build (dependencies, build, runtime) on the Bun/Debian base—Debian because Prisma's query engine requires glibc and OpenSSL—

producing a Next.js standalone server executed by an unprivileged user, with the SQLite file on a mounted volume so that levels, runs, and telemetry survive upgrades. The entrypoint applies the schema idempotently and boots; first boot imports and certifies the ladder (tens of seconds of CPU), and the container's healthcheck is the models endpoint. An optional Caddy sidecar terminates TLS.

11 Results and Observations

The shipped laboratory contains 12 certified levels and 13 recorded runs (265 persisted steps). The exhaustive baseline won all 12 levels at exactly the certified optimum, and every run was checker-certified—a round-trip proof that engine, validator, recorder, and replayer agree.

Table 4: Recorded runs at time of writing (excerpt).

Level	Agent	Outcome	Moves	Steps	Checker
01–12 (all)	wyrm-bfs	won	$= m^*$ each	$= m^*$	CERTIFIED
07 Key Note	glm-4.6	aborted[†]	23	30	CERTIFIED

[†]The provider API rate-limited mid-run; the 23 valid moves and all 30 recorded thoughts remain, illustrating the system's honest handling of infrastructure failure—the record is preserved, the run is marked, nothing is fabricated.

The partial LLM run is instructive as a specimen of what the instrument captures. In its recorded reasoning the model correctly parsed the key–lock relationship and planned the detour, but its step-by-step thoughts reveal it re-deriving the board geometry from the ASCII every turn rather than maintaining a stable internal map—visible thrash that the turn budget is designed to punish. We resist drawing quantitative conclusions from a single run; the system's purpose is to produce many such runs cheaply and auditably.

12 Related Work

Game environments have anchored AI evaluation since chess and Go [2, 5], through Dota 2 and StarCraft II [3, 4], with planning and search as the classical toolkit [6] and self-play reinforcement learning as the modern one [7]. WYRM.LAB's game mechanics descend from sokoban-style pushing puzzles and add families chosen specifically for their hostility to pattern-matching. For language models, static reasoning benchmarks [1, 8] probe single-step answers; planning studies [9] document systematic failures that game arenas can expose longitudinally. Agent architectures with self-critique and memory [10, 11] supply the design of our Reviewer and coach-note loop, and the Model Context Protocol [12] popularized the standardized tool surfaces our provider adapter layer deliberately avoids needing. Elo rating [13] remains the canonical comparative scale for pairwise encounters and is used unmodified here.

13 Limitations and Future Work

The benchmark's ladder is small (twelve levels) and its LLM run corpus is currently thin; both scale linearly with operator effort thanks to the Forge and the LLM level generator, but breadth awaits sustained cycles. SQLite and a single process bound the deployment to one node, which the determinism of the engine makes painless to outgrow (a run recorded anywhere replays anywhere). The difficulty score is structural and agent-agnostic by construction; an empirical calibration against a population of models—the score's D versus observed solve rates—is future work, as are multimodal variants in which the model receives the engraved rendering instead of

ASCII, and a formal analysis of the ice-slide mechanic as a minimal test of multi-step forward simulation.

14 Conclusion

WYRM.LAB demonstrates that a benchmark can be an instrument rather than a leaderboard: a deterministic game whose mechanics are engineered to break pattern-matching, an isolation model that never trusts the agent, a control plane that lets humans experiment on runs in flight, and an experience layer that shows the machine's textual world and a human engraving of the same state side by side. Every claim in this paper is backed by an artifact in the shipped system—the certified ladder, the checker verdicts, the recorded thoughts—and the whole laboratory deploys as one container. The worm, as always, is the point: an agent that can keep a map of a maze in its head, spend its own body wisely, and respect the irreversibility of its choices is doing something worth measuring honestly.

References

[1] K. Cobbe et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[2] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[3] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 574:350–354, 2019.

[4] C. Berner et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

[5] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[6] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson, 2020.

[7] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[8] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35, 2022.

[9] K. Valmeekam, M. Olmo, S. Sreedharan, and S. Kambhampati. On the planning abilities of large language models. *arXiv preprint arXiv:2305.15771*, 2023.

[10] S. Yao et al. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

[11] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[12] Anthropic. Model Context Protocol. `https://modelcontextprotocol.io`, 2024.

[13] A. E. Elo. *The Rating of Chessplayers, Past and Present.* Arco Publishing, 1978.